ZEROOTH REVIEW PRESENTATION

ChainLens:

Explainable Ethereum Wallet Risk Scoring

A bounded-window, rule-based pipeline for auditable crypto-risk signals

BLOCKCHAIN ANALYTICS · DATA MINING · FORENSIC RISK SCORING

GUIDE

Mrs. Parvathy E N
Assistant Professor
Artificial Intelligence and Data Science

TEAM MEMBERS · GROUP 11

Christo Philip Mathew	MUT24CA031
James S Alapatt	MUT24CS081
Nevin Alex Koshy	MUT24CS115
Richard Shaji Mekkaden	MUT24CS127

Muthoot Institute of Technology & Science (Autonomous), Kochi

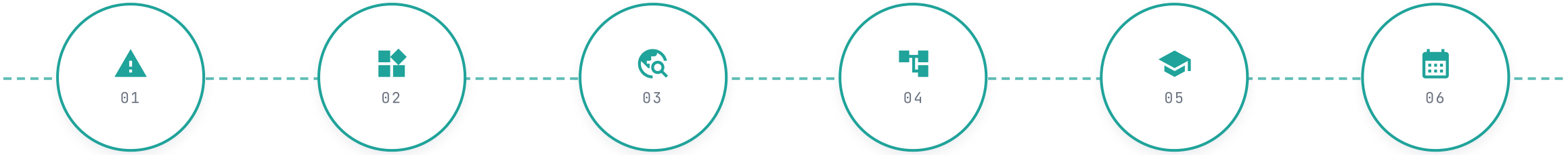


AGENDA

What We'll Cover Today

A six-step walkthrough — from problem to timeline.

6 SECTIONS



01 **Why this matters —** We frame the Ethereum-scale problem and the limits of black-box scoring, then state five concrete objectives.

02 **Foundations —** Cryptocurrency, blockchain anatomy, why money has value, and the dual-edged transparency of public ledgers.

03 **What we're building —** The ChainLens 7-stage pipeline, leakage-safe methodology, and bounded-window data collection.

04 **How it works —** Rule-based risk scoring (R1–R5), score_trace auditability, and the engineering stack behind it.

05 **Academic mapping —** Course outcomes, syllabus coverage, and alignment with UN SDG 16 on illicit financial flows.

06 **Timeline & next steps —** A 14-week build plan, deliverables per phase, and what we plan to demonstrate next.

THE PROBLEM

Problem Statement

! SCALE · OPACITY · JUSTIFICATION GAP

> Ethereum's public transaction data is valuable for compliance and security analysis, but its scale of millions of addresses and billions of transactions makes it hard to identify high-risk wallets tied to fraud, sanctions, or illicit activity. Existing tools compound this by producing opaque, black-box risk scores with no auditable evidence, leaving analysts unable to justify their decisions.

Scale
Millions of addresses, billions of transactions

Opacity
Black-box scores with no auditable evidence

Justification Gap
Analysts cannot justify their decisions

WHAT WE SET OUT TO ACHIEVE

Objectives

Five concrete goals for the ChainLens platform.

5 OBJECTIVES · R1–R5

1

Collect & analyze Ethereum transaction data

Within a bounded time window using publicly available sources.

2

Design explainable rule-based risk scoring

Implement an explainable, rule-based risk scoring system (R1–R5) for identifying high-risk wallets.

3

Generate auditable score_trace

For each wallet with transparent evidence and no black-box models.

4

Leakage-safe evaluation

Evaluate the effectiveness using a leakage-safe train/test split with metrics such as precision, recall, F1, PR-AUC.

5

User-friendly web application

Display wallet risk scores, transaction insights, and supporting evidence for better decision making.

PROJECT OVERVIEW

What We're Building

Five pillars of the ChainLens platform.

4 PILLARS + 1 GOAL

Data Analysis

Analyzes publicly available Ethereum transaction data to identify high-risk wallets involved in fraud, sanctions, or other illicit activities.

Risk Scoring System

Provides an explainable, rule-based risk scoring system (R1–R5) with transparent evidence instead of black-box models.

Interactive Dashboard

Offers a user-friendly dashboard for compliance analysts, researchers, and student auditors to explore wallet risk scores and transaction insights.

Auditability

Generates an auditable score_trace for each wallet with full traces of the contributing rules and signals.

Goal

A working prototype that identifies and ranks the top-N high-risk wallets within a bounded time window (or label-anchored subgraph).

Cryptocurrency & Blockchain

APPEND-ONLY LEDGER

Cryptocurrency
A digital medium of exchange secured by cryptography, not by a bank.

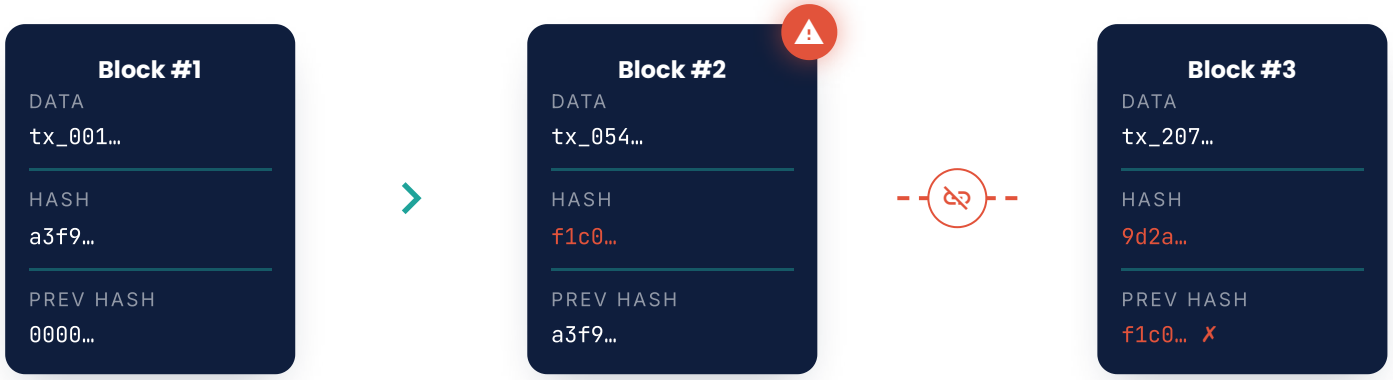
Block
A batch of verified transactions.

Chain
Each block stores the cryptographic hash of the previous block.

Result:
An append-only, tamper-evident ledger.

> *"Changing one old block is like pulling a single brick out of a tower — everything above collapses."*

TAMPERING BREAKS THE CHAIN



Intact link

Tampered hash — cascade invalidation

Why Does a Currency Have Value?

Fiat relies on institutional trust. Crypto relies on scarcity, utility, and protocol rules.

FIAT VS CRYPTO

FIAT CURRENCY	
Central Issuer	Trust in a government or central bank.
Legal Tender	Status enforced by law within a jurisdiction.
Institutional Backing	Backed by the credibility of the issuing state.

CRYPTOCURRENCY	
Scarcity	Fixed or capped supply (e.g., Bitcoin's 21M limit).
Utility	Paying for computation ("gas"), smart contracts, on-chain services.
Network Trust	Trust in protocol rules, not institutions.

Centralized — trust in a single issuer

Decentralized — trust in protocol rules

Fiat currency — government-issued money declared as legal tender; its value derives from the authority of the issuing state rather than from a physical commodity.

The Catch: Dark Side of Decentralization

TWO-EDGED TRANSPARENCY

No central authority

No central party is responsible for policing the network.

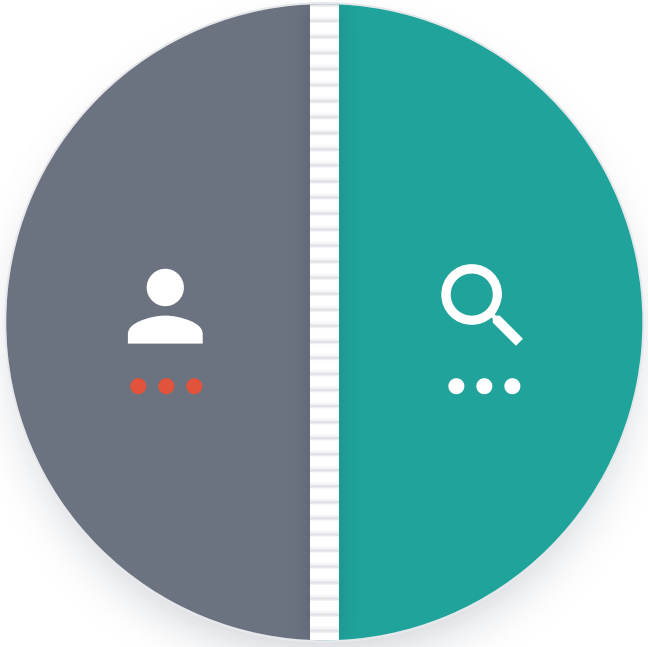
Attracts bad actors

Money launderers, ransomware operators, and scammers exploit the pseudonymous rails.

But it cuts both ways:
every transaction is permanently public.

Forensics is possible

The same transparency that enables crime also enables investigation.



Same ledger. Two uses.

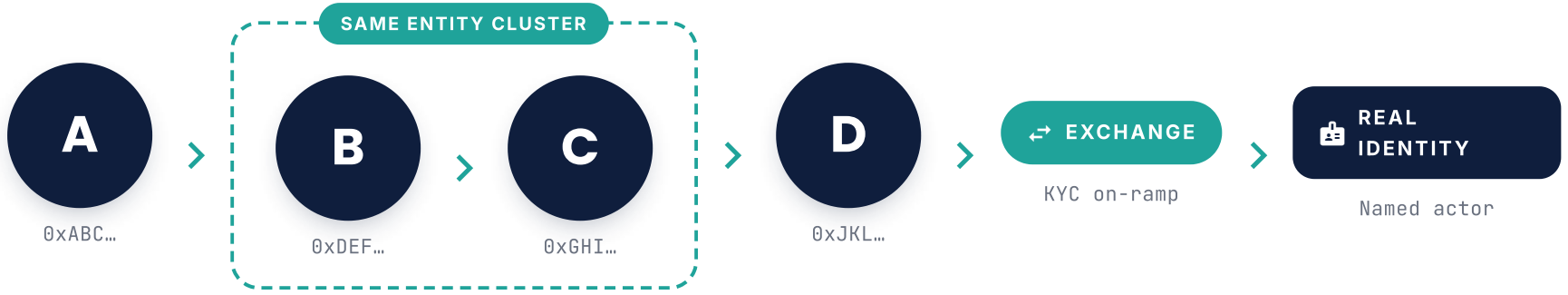
CORE CONCEPT

What is Chain Analysis?

Blockchain forensics — following the money across a permanent, public ledger.

FORENSICS

FOLLOW THE MONEY — 4-HOP TRACE



Permanently visible

Every public blockchain transaction is visible to anyone, forever.

Follow the money

Tracing fund flows across multiple hops.

Address clustering

Grouping addresses controlled by the same entity.

Identity linking

Connecting pseudonymous wallets to real-world identities via exchanges or leaked data.

PLATFORM CHOICE

Why Ethereum Specifically?

Three reasons Ethereum is the right sandbox for a low-budget analytics pipeline.

₿ ETHEREUM

01

EVM Standard

The execution model behind L2s, sidechains, and many independent chains — pipeline transfers to other EVM-compatible networks with only modest changes.

TRANSFERRABLE

02

Better Tooling

Mature ecosystem — Etherscan V2 API, NetworkX, DuckDB — ideal for a low-budget setup with off-the-shelf components.

OFF-THE-SHELF

03

Richer Transaction Data

Smart-contract interactions produce far more behavioral signals than simple value transfers — function calls, token transfers, event logs.

SIGNAL-DENSE

Result: a pipeline that is portable, well-tooled, and rich in signal.

SYSTEM ARCHITECTURE

The 7-Stage Decoupled Pipeline

DECOUPLED

Raw Ethereum data

1 **Ingestion**
Etherscan V2 API + OFAC SDN labels + sample fixtures.

↓

2 **Feature Engineering**
14 behavioral features per entity; log1p scaling fit on train only.

↓

3 **Detection / Signals**
R1–R5 rules + DBSCAN + Louvain community detection.

↓

4 **Risk Scoring**
Additive R1–R5 with 0.3x downweight, clamped 0–100.

↓

5 **Evaluation**
Precision, Recall, F1, PR-AUC + per-rule ablation.

↓

6 **Artifacts & Storage**
Parquet + JSON outputs (features, scores, traces, run_info).

↓

7 **Read-Only Dashboard**
Streamlit + Plotly, never imports pipeline code.

Analyst dashboard

Designed for Explainability

The pipeline is intentionally decoupled — every stage produces an auditable artifact. The dashboard never imports pipeline code, ensuring strict separation between computation and visualization.

AUDITABLE REPRODUCIBLE TRACEABLE

RUN_INFO.JSON

```
git_hash: a1b2c4d rules_sha: bb9b9712… scaler_sha: d7662267…
split_seed: 0x7c1b…
```

METHODOLOGY

End-to-End Workflow

Seven steps from raw Ethereum data to a read-only analyst dashboard, with a feedback loop for rule ablation.

7 STEPS

1 **Data Ingestion**
Etherscan V2 API + sample fixtures

2 **Label & Split**
OFAC SDN, MEW darklist, Kaggle fraud; SHA-256 split

3 **Feature Engineering**
DuckDB query, log1p scaling on train

4 **Detection & Scoring**
DBSCAN, NetworkX, Apriori, YAML R1–R5

5 **Evaluation & Presentation**
P/R/F1, PR-AUC, Streamlit dashboard

Step 6: Storage (Parquet/JSON)

`wallet_features.parquet` `scores.parquet` `traces.parquet` `run_info.json`

Step 7: Deployment / Execution

```
> uv run run.py --mode sample|real
```

No Docker, Makefile, or external orchestration.

FEEDBACK (PER-RULE ABLATION)

Evaluation → R1–R5 tuning

STORAGE DECISION

Storage — Parquet / JSON Only

Lightweight, file-based storage. No databases required.

FILE-BASED

Apache Parquet	JSON
Columnar binary format Efficient storage for large transaction datasets, features, scores, run artifacts.	**Human-readable metadata** Run information, config snapshots, version hashes.
PyArrow-backed Native Python read/write with exact DECIMAL(38,18) precision for ETH values (avoids float64 loss).	**Apriori rules, DBSCAN summary, Louvain communities** Structured intermediate outputs for audit and inspection.
Intermediate storage transactions, wallet_features (scaled), scores, score_trace.	**Reproducibility** run_info.json captures git hash, package versions, parameters.

NO DATABASES — uses Parquet/JSON only — no PostgreSQL, Neo4j, or other databases.
This keeps the pipeline lightweight and reproducible with a single command.

ARTIFACTS: `wallet_features.parquet` `graph_edges.parquet` `scores.parquet` `traces.parquet` `evaluation.json` `run_info.json`

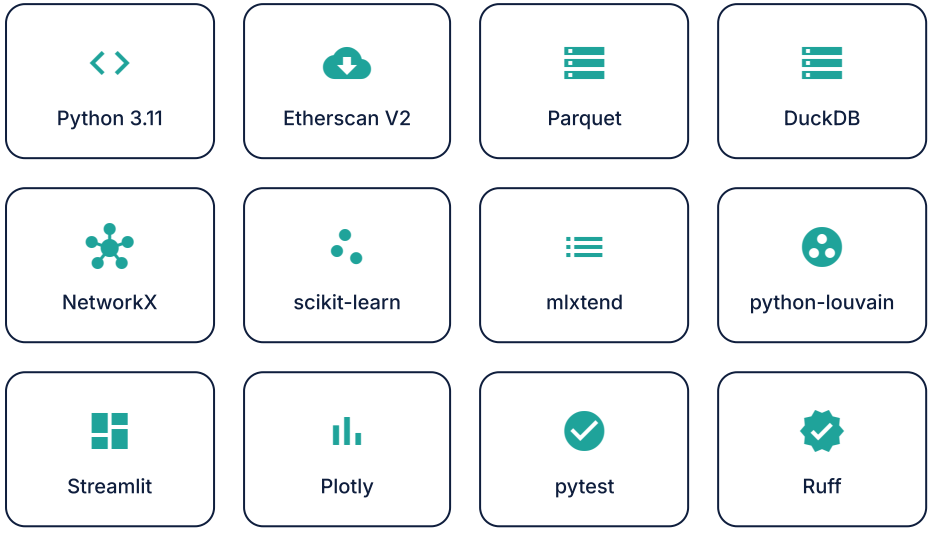
TECH STACK

Tooling & Technology

Open-source, off-the-shelf components — no proprietary infrastructure.

11 TOOLS

QUICK REFERENCE



Zero licensing cost

Every layer is free, open-source, and installable via `uv` or pip.

LAYER	TECHNOLOGY	PURPOSE
Programming Language	`Python 3.11`	Main language for data processing, ML, graph analysis, backend
Data Ingestion	`Etherscan V2 API (requests)`	Fetch Ethereum transaction data (or sample fixtures)
File Format & Storage	`Apache Parquet (PyArrow)`	Efficient columnar storage for transactions, features, scores
Query & Feature Eng	`DuckDB`	Fast analytical queries on Parquet with DECIMAL(38,18) precision
Graph Analysis	`NetworkX`	PageRank, SCC cycles, 1-hop neighborhood
Anomaly Detection	`scikit-learn (DBSCAN)`	Unsupervised clustering for anomalous wallets (noise points)
Association Rules	`mlxtend (Apriori)`	Mine association rules from Boolean features (train only)
Community Detection	`python-louvain`	Detect wallet communities (non-scoring, visualization)
Visualization / UI	`Streamlit + Plotly`	Read-only dashboard for risk scores and network visualization
Testing	`pytest`	Unit and integration tests for correctness and reproducibility
Code Quality	`Ruff`	Fast linting and formatting for maintainable code

RISK SCORING

Rule-Based Scoring (R1–R5)

Weights locked in a YAML config and recorded with SHA-256 hash for reproducibility.

R1 · R2 · R3 · R4 · R5

R1 · Cycle Member

Part of a transaction cycle (3–5 wallets, value/time constraints).

25%

R2 · Fan-out

Sends to many distinct recipients (high fan-out).

20%

R3 · Sanctioned 1-hop

Directly connected to a known sanctioned address.

30%

R4 · High PageRank

Top 5% of PageRank in the transaction graph.

15%

R5 · Apriori Match

Matches a learned association rule (train only).

20%

```
Final = (0.25·R1 + 0.20·R2 + 0.30·R3 + 0.15·R4 + 0.20·R5) × downweight, clamped to 100
```

Known-address downweight 0.3× if applicable (70% reduction for benign entities like exchanges/DeFi).

TIER	SCORE RANGE	INTERPRETATION
Low	0–24	No significant risk signals
Medium	25–49	Some risk indicators present
High	50–74	Multiple risk indicators
Severe	75–100	Strong risk indicators

ACADEMIC MAPPING · 1 OF 3

Syllabus → Project

Every course unit maps to a concrete module in the ChainLens pipeline.

4 UNITS

CURRICULUM COVERAGE

UNIT 1

UNIT 1

KDD & Warehousing

UNIT 2

UNIT 2

Preprocessing

CL

UNIT 3

UNIT 3

Clustering

UNIT 4

UNIT 4

Association & Graph Mining

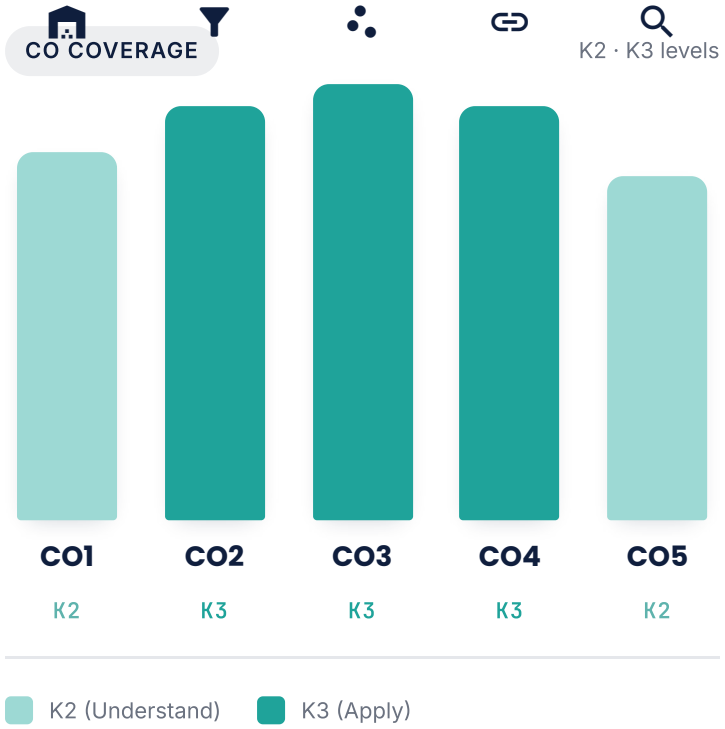
Four units, one coherent pipeline — ChainLens spans the full KDD lifecycle.

UNIT → MODULE MAPPING

SYLLABUS UNIT	WHERE IT SHOWS UP
UNIT 1 **KDD & Data Warehousing**	Pipeline framed as KDD process; Parquet/JSON artifact schema
UNIT 2 **Data Preprocessing**	Cleaning raw txns (receipt_status=1, Wei→ETH), SHA-256 split, feature scaling
UNIT 3 **Classification & Clustering**	DBSCAN on wallet behavior; P/R/F1 evaluation
UNIT 4 **Association Rule Mining & Advanced DM**	Apriori feeding R5; PageRank feeding R4

Course Outcome Mapping

Every CO is exercised somewhere in the ChainLens build.



CO	DESCRIPTION	WHERE IN PROJECT
CO1 K2	Understand KDD & data warehousing	**Pipeline as KDD process; Parquet/JSON artifact schema**
CO2 K3	Apply preprocessing on raw data	**Cleaning Ethereum txns (receipt_status=1, Wei→ETH), SHA-256 split**
CO3 K3	Illustrate classification & clustering	**DBSCAN clustering of wallet behavior; P/R/F1 evaluation**
CO4 K3	Comprehend association rule mining	**Apriori feeding the R5 rule in the scoring engine**
CO5 K2	Explain advanced DM in emerging domains	**Graph mining (PageRank, SCC) applied to crypto forensics**

Five course outcomes — three at K3 (Apply) and two at K2 (Understand) — every CO has a concrete traceable artifact in the ChainLens pipeline.

ACADEMIC MAPPING · 3 OF 3

Sustainable Development Goals

Where ChainLens fits in the UN SDG framework.

PRIMARY FIT

SDG 16

Peace, Justice & Strong Institutions

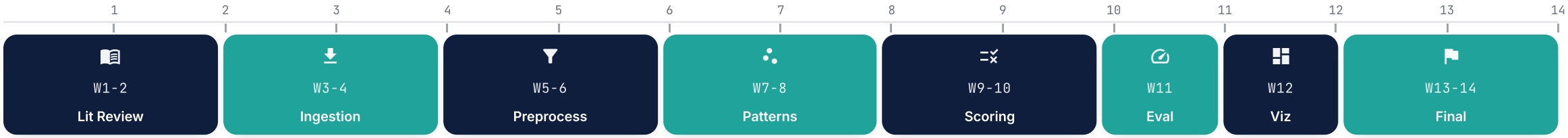
- Combating money laundering, fraud, and terrorism financing through auditable risk scoring
- Improving institutional transparency and accountability in crypto markets
- Enabling law enforcement with forensic-grade, explainable evidence (`score_trace`)

GLOBAL RELEVANCE Directly supports the SDG 16 target of reducing illicit financial flows and strengthening the recovery and return of stolen assets.

PROJECT PLAN

14-Week Project Timeline

Eight milestones across fourteen weeks — built-in buffer for final review prep.



WEEKS	MILESTONE
1–2	Literature review, finalize problem, pick data source, Review 0
3–4	Ingestion pipeline (Etherscan V2 + sample fixtures)
5–6	Feature engineering (DuckDB); SHA-256 split; storage schema
7–8	Detection / pattern mining (DBSCAN, SCC, PageRank, Apriori)
9–10	Rule-based scoring engine (R1–R5 with YAML + SHA-256)
11	Evaluation against labelled addresses (P/R/F1, PR-AUC)
12	Visualization / dashboard, documentation
13–14	Final review preparation, buffer

APPENDIX · DESIGN CHOICE

Design Choice: DBSCAN Over K-Means

Three concrete reasons DBSCAN fits anomaly detection better than K-Means.

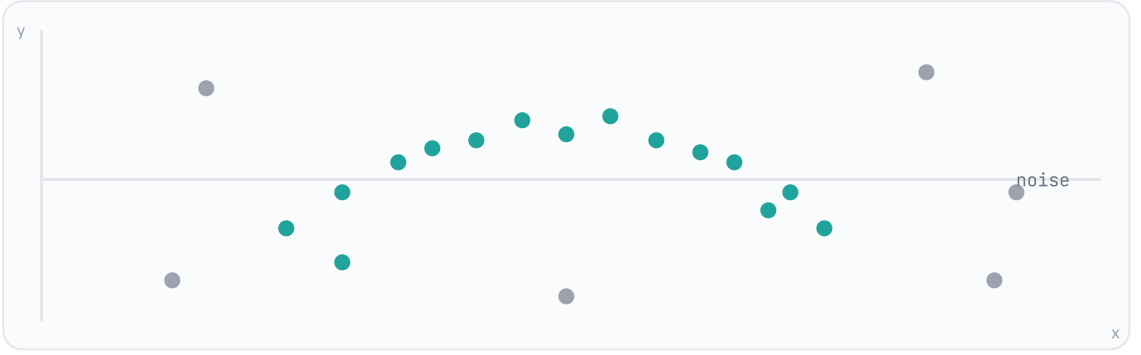
CONSIDERATION	✕ K-MEANS	✓ DBSCAN
Specify cluster count?	Yes (k) — but we don't know how many behavior types exist	No — discovers clusters automatically
Handles outliers/noise?	No — forces every point into a cluster	Yes — labels outliers as noise (ideal for anomaly detection)
Cluster shape	Assumes spherical clusters	Finds arbitrarily shaped clusters

Verdict: Wallet behavior clusters are non-spherical and the unknown shape/count of illicit patterns makes DBSCAN a natural fit. K-Means' spherical assumption would silently merge anomalous wallets into "normal" clusters.

K-MEANS — spherical, forced



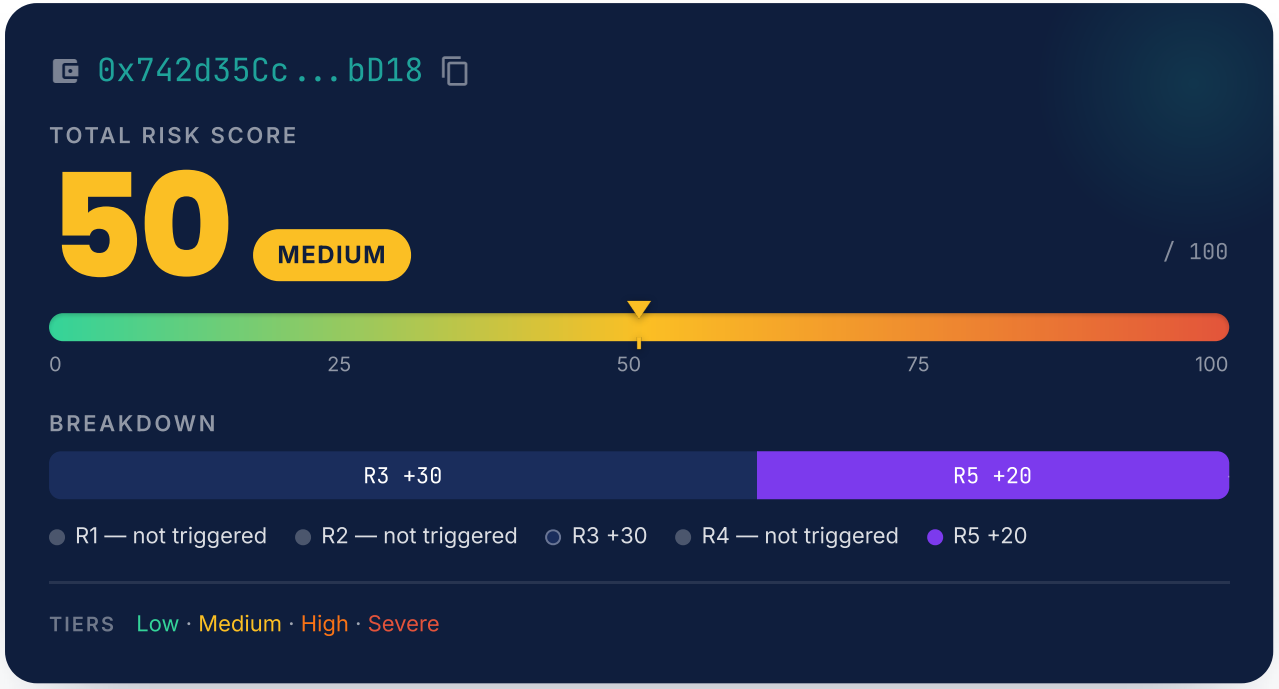
DBSCAN — shape-aware, noise-aware



EXPECTED OUTPUT

Explainable Risk Score — Sample

How the `score_trace` will decompose a final score back into triggered rules.



Address: 0x742d35Cc...bD18 Total Risk Score: 50 (Medium) Breakdown: *R1 Cycle member — not triggered R2 Fan-out — not triggered* R3 Sanctioned 1-hop +30 *R4 PageRank top 5% — not triggered* R5 Apriori match +20 Known-address downweight: 1.0× (not applicable) Clamped to: 50 / 100

INTERPRETATION

This wallet shows (1) a direct 1-hop link to a sanctioned address and (2) a learned behavioral association pattern. Together these warrant a **Medium tier (25–49)** for follow-up forensic review.

References

Data sources, libraries, and foundational works used in the ChainLens implementation.

1. Etherscan. *Etherscan API V2 Documentation — Account and Transaction Endpoints.* *(accessed Oct 2026).* DATA SOURCE
2. U.S. Department of the Treasury. *Specially Designated Nationals (SDN) List — Digital Currency Addresses.* *(accessed Oct 2026).* LABELS
3. MyEtherWallet. *ethereum-lists — Community Darklist of Malicious Addresses.* GitHub. LABELS
4. NetworkX Developers. *NetworkX Documentation — PageRank, Strongly Connected Components, Cycle Detection.* *(accessed Oct 2026).* GRAPH ANALYSIS
5. Ester, M., Kriegel, H.-P., Sander, J., Xu, X. *A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise.* KDD 1996. DBSCAN
6. Agrawal, R., Srikant, R. *Fast Algorithms for Mining Association Rules.* VLDB 1994. APRIORI
7. Python-Louvain. *Louvain Community Detection for NetworkX Graphs.* *(accessed Oct 2026).* COMMUNITY
8. DuckDB Foundation. *DuckDB Documentation — Parquet and DECIMAL Types.* *(accessed Oct 2026).* QUERY
9. Streamlit. *Streamlit Documentation — Multipage Apps and Dataframes.* *(accessed Oct 2026).* UI
10. Plotly. *Plotly Python Documentation — Charts and Network Visualisation.* *(accessed Oct 2026).* UI
11. Pedregosa, F. et al. *Scikit-learn: Machine Learning in Python.* Journal of Machine Learning Research, 12, 2825–2830, 2011. SCIKIT-LEARN
12. Raschka, S. et al. *MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack.* Journal of Open Source Software, 3(24), 638, 2018. MLXTEND
13. Apache Arrow. *PyArrow and Apache Parquet Documentation.* *(accessed Oct 2026).* STORAGE
14. Astral. *uv Documentation — Python Project and Environment Management.* *(accessed Oct 2026).* TOOLING
15. pytest Developers. *pytest Documentation — Testing Python Code.* *(accessed Oct 2026).* TESTING

{} **NOTE:** *Include only data sources actually loaded by the implementation.*



Thank You

An explainable lens on Ethereum wallet risk.

Ethereum Analytics | Explainable Scoring | Forensic-Grade